

GENAI BOOTCAMP · COMPLETE STUDY NOTES

RAG

Retrieval-Augmented Generation — From Fundamentals to Interview Prep

RAG Pipeline

Data Parsing

Chunking

Vector Databases

Retrievers

Query Enhancement

Reranking

Interview Q&A

Table of Contents

1. **01** Introduction to RAG — What, Why & Pipeline
2. **02** Data Sources & Data Parsing
3. **03** Chunking — Strategies & Best Practices
4. **04** Vector Databases — Storage & Indexing
5. **05** Retrievers — Core Concepts & Search Types
6. **06** Metadata Filtering, Sparse/Dense/Hybrid Retrieval
7. **07** Query Transformation & Enhancement (Expansion, Decomposition, HyDE)
8. **08** Reranking, MMR & Result Fusion
9. **09** Advanced Retrieval Patterns (Parent Doc, Sentence Window, Multi-Hop, Multimodal)
0. **10** Contextual Compression
1. **11** RAG vs Fine-Tuning
2. **12** LangChain Quick-Reference (Classes & Imports)
3. **13** Interview Preparation — Q&A

Introduction to RAG

What is RAG?

DEFINITION

RAG stands for **Retrieval-Augmented Generation** — a technique that helps an LLM answer questions using external data such as PDFs, documents, websites, databases, or company knowledge bases. In short: **give relevant context to the LLM before it answers.**

- **Retrieval** = finding relevant information from documents
- **Augmentation** = adding that retrieved information as extra context to the user's question (improving the LLM's input)
- **Generation** = the LLM generates the final answer using that context

```
Large Document / Knowledge Base
↓
Data Ingestion (Parsing → Chunking → Embedding)
↓
Vector Database (stores embeddings)
↓
User Query → Query Embedding → Similarity Search → Top-K Retrieved Chunks
↓
Prompt = Question + Retrieved Context
↓
LLM
↓
Final Answer
```

Worked Example

EXAMPLE

User asks: "What is the refund policy of our company?"

The LLM may not know this because company policy is private data. RAG does this:

1. Searches the company policy document
2. Retrieves the relevant refund policy section
3. Adds that retrieved section as extra context to the user question
4. Gives the context + question to the LLM
5. The LLM generates the final answer based on that context

Retrieved context: "Refunds are allowed within 7 days of purchase." → **Augmented prompt:** Question: What is the refund policy? / Context: Refunds are allowed within 7 days of purchase. → **Final answer:** The refund policy allows refunds within 7 days of purchase.

Why RAG? (Problems It Solves)

Problem	Explanation
1. LLMs do not have private data	A model cannot know internal/company-specific or proprietary documents unless given as context.
2. LLMs may not have the latest data	Training data has a cutoff; recent events or updated policies aren't known.
3. LLMs can hallucinate	Sometimes LLMs generate confident but wrong answers. Example: user asks about ABC Company's refund policy; without RAG the LLM may confidently say "30-day policy" when the real document says "7 days only" — a hallucination. RAG reduces this because the answer is grounded in retrieved context.
4. Fine-tuning is not always the right solution	For every new document we cannot fine-tune the model again and again. RAG is better when data changes frequently — e.g., if an HR policy changes every month, update documents in RAG rather than fine-tuning monthly.

LLMs do not have access to the latest/private data
↓
We retrieve relevant context from documents
↓
We give the LLM the context + question
↓
The LLM gives a grounded answer

Pipeline to Implement RAG

RAG helps an LLM answer questions using external documents. First, we parse the documents and extract clean text. Then we split the text into chunks, convert chunks into embeddings, and store them in a vector database. When the user asks a question, we retrieve the most relevant chunks and give them to the LLM along with the question.

INGESTION / INDEXING
Documents/Data
→ Chunking (optional)
→ Embedding
→ Vector Database (Vector Store)

RETRIEVAL / GENERATION
User Query → Query Embedding
→ Similarity Search
→ Top-k Retrieved Chunks (Top 3)
→ Prompt/Context (Query + Chunks)
→ LLM → Answer

3 Core Steps of RAG

1. **Retrieval** — vector DB se relevant docs find karna
2. **Augmentation** — context ka metadata se enrich karna
3. **Generation** — LLM se final answer banana

Core Components of RAG

- 1) Document Ingestion Phase
PDF / Web / DB → Document Splitter → Chunk1, Chunk2, Chunk3 → Embedding Model → Vector Database (FAISS / Pinecone)
- 2) Query Processing Phase
User Query → Query-to-Embedding → Similarity Search (cosine similarity, k-NN)
→ Find most relevant chunks (Relevant Chunk 1, 2, 3...) → top-k
- 3) Generation Phase
Augmented Context (query + chunk) → LLM → Generate Response → User

RAG's 10 Moving Components

RAG "seems" very simple, but it has many moving components — a moving component is one whose output directly contributes to the end result:

1. Data Parsing
2. Chunking
3. Embedding
4. Vector DB
5. LLM
6. Graph RAG (alternative)
7. Retriever
8. Prompt arrangement
9. Which LLM to use
0. Output Parser

Alternative Storage Paradigms

Approach	Flow
Vector-based (most common)	Data → embed → Vector Store (Local: in-memory/on-disk, or Cloud: 3rd-party/native)
Rule-based	Data → Rules → Vector-ley (structured/rule indexing)

Vector Database → Simple Example

why require a vector database?

Traditional DB → exact match only

Vector DB → meaning-based match

e.g. searching "cat" (converted to vector) can find "kitten" because the meaning vectors are close – traditional DBs cannot do this.

Code Reminder — LCEL RAG Chain

```
rag_chain = (  
    {"context": retriever | format_docs, "question": RunnablePassthrough()}  
    | prompt  
    | llm  
    | StrOutputParser()  
)
```

`retriever` pulls from the VectorDB → `format_docs` extracts `document.page_content` from each Document object → `prompt / llm` generate the answer → `StrOutputParser()` converts the LLM output object into a plain string.

Data Sources & Data Parsing

Data Can Be Understood from 3 Perspectives

1. Nature / Modality	2. Storage Format	3. Structure
Text, Image, Audio, Video	PDF, DOCX, CSV, JSON, YAML, MP3, MP4	Structured — Tables, CSV, SQL Semi-structured — JSON, YAML, HTML Unstructured — PDFs, Images, Audio, Video, Reports

Common Data Sources for RAG

PDF DOCX Internal DB DB Web Internal Portal Cloud Architecture Audio / Image / Video

Document AI & Parsing Tools

Free	Paid / Expensive
LlamaParse, Docling, python-docx, PyMuPDF, pdfplumber, Parsio, Unstructured.io	Google Document AI, Amazon Textract, Azure Document Intelligence (ADI)

Paid OCR/parsing services (ADI, Amazon Textract, GCP Document AI) are generally more accurate for scanned/complex layouts but are expensive — often crossed out as "too costly" for routine pipelines in favour of free open-source parsers.

Web → HTML → `<img>` `<table>` `<text>`
 Data Format / File Format examples: PDF, DOCX, Appended docs
 Tabular file formats: CSV, TSV, XLSX → arranged as Tabular data

LangChain's Role in Parsing

LangChain does provide document loaders, but in most cases LangChain is **not** doing the low-level parsing itself. It mainly provides a common wrapper/interface over different parsing libraries and services.

- LangChain itself is not primarily a parsing library.
- LangChain's document loaders internally use existing parsing libraries such as PyPDF, PyMuPDF, pdfplumber, python-docx, Unstructured, and others.
- LangChain acts like a wrapper/integration layer that gives a common Document format: `page_content + metadata`.
- This makes it easy to plug parsed documents into the RAG pipeline.

LangChain "7 Building Blocks" (from notes)

1. Document Loader / Document Parser
2. Chunking method
3. Vector DB (VDB)
4. Model / LLM / API (OpenAI, Claude, Gemini, HF, Ollama)
5. Cloud
6. Memory management
7. Chaining / LCEL

Note: since LangChain is not having its own parser, its Document Loader acts as a wrapper around: PDFPlumber, PyMuPDF, PyPDF, Unstructured.io, python-docx, etc.

Vector Databases & Search (RAG Backbone)

FAISS Chroma Qdrant Supabase pgvector Milvus Pinecone AWS OpenSearch Azure AI Search

LangChain wraps different parsing libraries so developers can use many data sources through one common interface. Its goal is not only parsing, but connecting parsing with chunking, embeddings, vector databases, retrievers, prompts, and LLMs — making the full RAG pipeline easier to build and maintain.

Chunking

What is Chunking?

Chunking is the process of dividing large data into smaller, meaningful units called **chunks**. In a RAG pipeline, each chunk can be embedded separately, stored separately, retrieved separately, and sent to an LLM only when relevant.

```
Large document
↓
Chunking
↓
Chunk 1, Chunk 2, Chunk 3...
↓
Embeddings and retrieval
↓
Relevant chunks sent to the LLM
```

Simple Example

Suppose a document contains three topics: (1) What is artificial intelligence?, (2) What is machine learning?, (3) What is deep learning?. Instead of storing the whole document as one unit, we create: Chunk 1 → AI section, Chunk 2 → ML section, Chunk 3 → DL section.

Analogy — Phone Number Chunking

Think of a 10-digit phone number **8770203258**. If someone asks for it, chunking it into groups (2-digit: 87-70-20-32-58, or 3-digit: 877-020-3258) changes how easily it's recalled/grouped — similarly, chunk size changes how information is grouped for retrieval.

Why Do We Need Chunking?

Reason	Explanation
LLM context-window limitations	Every LLM has a maximum input capacity; a very large document may not fit inside one request (e.g. 4096-token limits). Sending all 55 pages of a PDF at once = too much data stuffed in one context.
Better retrieval accuracy	A small relevant section is easier to retrieve than one huge document containing many unrelated topics.
Lower token usage & cost	Only relevant chunks need to be sent to the LLM.
Faster processing	Smaller units are easier to embed, index, retrieve, and process.
Sending only relevant information	The LLM receives focused context instead of the entire knowledge base.

IMPORTANT BALANCE

Very small chunks may lose context. Very large chunks may contain irrelevant information. Excessive overlap creates duplication and increases storage and token usage.

Is Chunking Always Required?

No. Chunking is a design choice, not a compulsory step.

USUALLY REQUIRED WHEN

- The document is large
- Document exceeds the model context window
- Semantic retrieval is required
- Different sections discuss different topics

MAY NOT BE REQUIRED WHEN

- Input is already short
- One complete record is already a meaningful unit
- SQL can retrieve exact rows
- Full document required for the task

- Only a small section should be sent to the LLM
- Source is a long PDF, manual, policy, or knowledge base

- Complete input comfortably fits the model context
- Data already separated into FAQs/tickets/products/DB records

Example: One FAQ question + answer → already an atomic unit → further chunking may not be needed. However, a 200-page policy PDF → chunking is generally required.

LangChain Splitter Classes

RecursiveCharacterTextSplitter TokenTextSplitter CharacterTextSplitter MarkdownHeaderTextSplitter HTMLHeaderTextSplitter
HTMLSemanticPreservingSplitter NLTKTextSplitter SpacyTextSplitter RecursiveJsonSplitter PythonCodeTextSplitter

4 Chunking Strategy Categories

1. **Fixed-size Chunking** — split by a fixed character/token count
2. **Content-aware Chunking** — preserves natural boundaries (paragraphs, sentences, headings)
3. **Document Structure-Based Chunking** — splits by layout (headers, sections)
4. **Semantic Chunking** — splits by meaning/topic-change using embeddings

Chunk Size & Chunk Overlap

Chunk size = the maximum target amount of content in one chunk (e.g. 300 characters / 300 tokens).

Chunk overlap = the amount of repeated content between neighboring chunks. It helps when an important sentence/concept falls near a chunk boundary, but excessive overlap increases duplication.

Chunk size and overlap are **hyperparameters** — we cannot decide the "correct" value analytically; it is experimental / evaluated using real questions and retrieval results.

- Smaller chunk size usually creates more chunks.
- Larger overlap usually increases repeated stored content.
- A larger chunk may preserve more context but reduce retrieval precision.
- A smaller chunk may improve precision but lose surrounding explanation.

Worked Example

SUPPOSE

Chunk size = 10 words, Chunk overlap = 3 words.

Original text: "Artificial intelligence helps machines learn patterns and make useful decisions from data."

Chunk 1: Artificial intelligence helps machines learn patterns and make useful decisions

Chunk 2: make useful decisions from data

The words "*make useful decisions*" repeat in both chunks — this repeated part is the **chunk overlap**. It helps preserve meaning when an important sentence is divided between two chunks.

Common Mistakes While Chunking

- Choosing chunk size without evaluating retrieval
- Creating chunks that are too small
- Creating chunks that are too large
- Adding excessive overlap
- Splitting headings away from their content
- Splitting tables without preserving headers
- Splitting code in the middle of functions or classes
- Removing useful metadata
- Using semantic or agentic chunking without comparing a simpler baseline
- Assuming one strategy works for every data type

Chunking Beyond Text

Chunking is not limited to text — it can also divide images, tables, code, audio, video, maps, medical scans, and document layouts into smaller meaningful units. The chunking boundary depends on the type of data.

Image Chunking

Dividing a large image into smaller visual regions. Can be divided using: fixed tiles, overlapping tiles, detected objects, document-layout regions, semantic regions.

Example: a scanned document image may be divided into Header, Paragraph, Table, Chart, Image, Footer.

Useful when: image is very large; small text may disappear after resizing; tiny objects need to be detected; image has many independent regions; region-level retrieval required; image is a map/medical scan/diagram/satellite image.

LIMITATION

Image chunking may lose the overall relationship between different regions. Therefore a strong system may store **both**: Full image → global context, and Image chunks → local details.

Table Chunking

Dividing a large table into smaller logical sections. Can be chunked by: rows, overlapping rows, columns, categories, entities, dates/time ranges.

- **Row-based:** Chunk 1 → Rows 1–3, Chunk 2 → Rows 3–5, Chunk 3 → Rows 5–7 (repeated rows = overlap). Useful for related nearby records (transaction history, logs, sensor readings).
- **Category-based:** Chunk 1 → Electronics products, Chunk 2 → Furniture products, Chunk 3 → Stationery products (group-by) — often more meaningful than splitting after an arbitrary row count.
- **Column-based:** Chunk 1 → Product info, Chunk 2 → Sales info, Chunk 3 → Location info; a common key (e.g. Order_ID) should be preserved in every chunk so chunks stay connected.

Every table chunk should preserve: column names, primary/joining keys, table title, units, relevant metadata, row & column meaning. Without column names, values become unclear — e.g. "O-101, Laptop, 2, 2400" is less meaningful than a labeled version: "Order ID: O-101 / Product: Laptop / Quantity: 2 / Revenue: \$2,400".

How table chunks are used in RAG: Since embedding models work better with descriptive text, a table chunk is often converted into text such as: "*Table: Sales Orders — Order ID: O-101, Product: Laptop, Category: Electronics, Region: North, Quantity: 2, Revenue: \$2,400*" — this text is then embedded, stored, and retrieved.

WHEN NOT TO EMBED A TABLE

When data is already in a SQL database and the user wants exact calculations, filters, joins, totals, or averages — SQL is usually the better solution (User question → SQL query → exact rows/calculated result → LLM explanation). Examples: "What is the total revenue?", "Which product generated the highest sales?", "Show all orders from the North region."

Vector retrieval is more useful when: the table comes from a PDF, is not stored in a database, semantic matching is required, the user asks meaning-based questions, or exact SQL querying is unavailable.

Semantic Chunking (Deep Dive)

PROBLEM WITH NAIVE/FIXED CHUNKING

A simple fixed-size splitter (e.g. FAISS-style 500-token cut) blindly cuts the document without regard for its idea/meaning — it may split a concept in half in the middle of an argument.

SOLUTION

Semantic chunking cuts based on where the **meaning** changes, not on a blind token count.

Semantic Chunking — 5 Step Pipeline

1. **Document Segmentation** — split document into sentences or paragraphs
2. **Sentence Embedding** — convert every sentence into a vector (e.g. via SentenceTransformer/OpenAI/HF)
3. **Semantic Similarity Check** — compute cosine similarity between adjacent sentences
4. **Merging** — merge adjacent sentences into the same chunk if they are similar
5. **Boundary Detection** — a topic-change boundary is placed where similarity drops below a threshold

```
semantic_chunking()
```

```

↓
create_sentence_windows()
↓
create_local_embeddings(windows)
↓
SentenceTransformer(model_name)
↓
model.encode(windows)
↓
embeddings
↓
similarity and distance calculation
↓
topic-change boundaries
↓
semantic chunks

```

WORKED NUMERIC EXAMPLE

S1 ↔ S2 similarity = 0.95 → **similar** → same chunk
 S2 ↔ S3 similarity = 0.30 (different) → **along** new chunk boundary
 Merging → merge adjacent sentence if they are semantically similar

Content-Aware Chunking (Comparison)

Content-aware chunking splits text by understanding its natural structure, instead of cutting blindly after a fixed number of characters or tokens. It tries to preserve boundaries such as paragraphs, sentences, headings, lists, sections, tables, code blocks.

Fixed-size splitter (may create)	Content-aware chunking (creates)
Chunk 1: "AI enables machines to perform intelligent tasks. Machine learning is a part of AI. Employee Leave" Chunk 2: "Policy. Employees receive 20 paid leave days..."	Chunk 1: "Artificial Intelligence — AI enables machines to perform intelligent tasks. Machine learning is a part of AI." Chunk 2: "Employee Leave Policy — Employees receive 20 paid leave days. Leave must be approved by the manager."

Code Reference — LangChain Splitters

RecursiveCharacterTextSplitter

```

recursive_splitter = RecursiveCharacterTextSplitter(
    separators=["\n\n", "\n", ". ", " ", ""],
    chunk_size=300,
    chunk_overlap=50,
    length_function=len,
    add_start_index=True,
)

```

It splits text recursively using multiple separators in priority order:

1. `"\n\n"` — tries paragraph boundaries first
 2. If a paragraph > 300 characters → `"\n"` — line boundaries
 3. If a line is still too large → `". "` — sentence boundaries
 4. If a sentence is still too large → `" "` — splits between words
 5. Final fallback: `""` — splits at the individual-character level
- `chunk_size=300` → final chunks should be approximately no more than 300 characters
 - `chunk_overlap=50` → ~50 characters from the previous chunk repeat in the next so context isn't lost
 - `length_function=len` → chunk size is calculated using the number of characters
 - `add_start_index=True` → stores the character position where each chunk starts, as metadata

This approach tries to preserve paragraph, line, sentence, and word boundaries — useful for general RAG applications.

TokenTextSplitter

Divides text according to **tokens**, not characters/words/paragraphs/sentences. `encoding_name="cl100k_base"` selects the tokenizer.

```
# chunk_size = 5 tokens, chunk_overlap = 2 tokens
text: "AI helps machines learn from data and make better decision for users"
tokens: [AI][helps][machines][learn][from][data][and][make][better][decisions][for][users]

Chunk 1: AI helps machines learn from
Chunk 2: learn from data and make
Chunk 3: and make better decisions for
Chunk 4: decisions for users
```

Chunking can be applied to text, images, tables, code, audio, and video. Image chunking divides a large image into smaller visual regions so text, objects, tables, and charts can be processed separately. Table chunking divides a large table by rows, columns, categories, entities, or dates, and must preserve headers/keys/units/metadata. However, chunking is not always the best option — for exact structured-data questions, SQL is usually better than vector retrieval.

Vector Databases

What is a Vector Database?

A vector database stores data as numerical vectors called **embeddings** and retrieves the most similar vectors using **similarity search**. It is mainly used for semantic search, RAG, recommendation systems, and image search — where results are found by **meaning** rather than exact keyword matching.

Why is a Vector Database Required?

Traditional databases perform exact matching. A vector database converts data into embeddings (numeric vectors) so that meaning-based matches can be found — e.g. searching "cat" can retrieve "kitten" because their meaning vectors are close, which a traditional DB cannot do.

What is Embedding?

Embedding = converting any type of data (text, image, audio, video) into a numeric vector, so that meaning/similarity can be compared mathematically. Text/Image/Audio/Video → Embedding → Vector/Numeric array `[ | | | | ]`.

What is a Vector Index?

A vector index is a data structure that organizes embeddings so the vector database can find similar vectors quickly, without comparing the query against every stored vector. Examples: **Flat**, **IVF**, **HNSW**. The vector database *stores* vectors; the vector index makes *searching* those vectors faster.

Inside the index, we store the **actual data** along with its **corresponding embedding (original data)** and **metadata**.

What is Similarity Search?

Similarity search is the process of finding stored vectors that are closest or most similar to a query vector. It compares vectors using metrics such as **cosine similarity**, **Euclidean distance**, or **dot product**.

EXAMPLE

A search for "How can I recover my account?" may retrieve "Steps to reset your password" because both have similar meaning, even though the words are different.

Similarity Search Formulas

Method	What it Measures	Best Result
Cosine Similarity	Angle / direction between two vectors: $\text{Cosine}(A,B) = (A \cdot B) / (\ A\ \ B\)$. Most commonly used for text embeddings.	Highest score = more similar
Euclidean Distance (L2)	Straight-line distance between two vectors	Lowest distance = more similar
Dot Product	Multiplies corresponding vector values and sums them. With normalized vectors, this becomes equivalent to cosine similarity.	Highest score = more similar

Core Types of Vector Similarity Search

Method	Accuracy	Speed	Best For
Exact Nearest Neighbor Search (checks query against every stored vector)	100% exact	Slower on large datasets	Small datasets, evaluation
Approximate Nearest Neighbor (ANN) (searches only a selected part of the index)	Approximate / high recall	Much faster	Large-scale production search

Examples: Exact search → Flat index. ANN search → HNSW, IVF. Exact search prioritizes accuracy; ANN prioritizes speed

and scalability. In production, ANN can occasionally miss the true best result, but the trade-off is worth the speed at scale (e.g. searching millions of rows via *linear search* going through all rows is very time-consuming — ANN avoids this).

IVF Index (Inverted File / Clustering-Based)

IVF divides all vectors into **clusters** (grouping similar vectors together). A query embedding first finds the **nearest cluster(s)**, and the search runs only within that/those cluster(s) — instead of comparing against every row. This reduces the search space and gives faster results.

All vectors → Cluster grouping (similar kind of data ends up in one cluster)
Cluster 1 ← AI-related data
Cluster 2 ← RL-related data
Cluster 3 ← DE-related data
...
Cluster N

Query → embedding (vector) → find nearest cluster
→ select top-k results from that cluster only
→ our search space reduces → faster result

HNSW (Hierarchical Navigable Small World)

HNSW is a graph-based, hierarchical, multi-layer navigable structure. The query starts at the top (sparse) layer and navigates down through denser layers, following the graph edges toward the nearest neighbors — this gives an **approximate** search that avoids scanning every vector, unlike a full linear scan.

Query → Find approximation over top layer → navigate down the graph layers → best matching nodes.

Approximation Techniques Summary

Clustering (IVF) Inverted File Index Hierarchical Navigable Small World (HNSW) / Graph

Vector Database Deployment Models

Model	Examples	Who Manages It
In-memory	FAISS, Chroma DB	You (fastest, not persistent by default)
Local persistent (local server)	FAISS, Chroma, Qdrant, Milvus	You manage everything (self-hosted)
Vendor-managed SaaS	Pinecone, Qdrant, Milvus, Weaviate	The database vendor manages it
Cloud-provider managed	Azure AI Search, AWS OpenSearch, Vertex AI Vector Search, BigQuery Vector Search	AWS / Azure / GCP manages it

SQL vs NoSQL vs Vector Database

Feature	SQL Database	NoSQL Database	Vector Database
Primary purpose	Structured relational data and transactions	Flexible and distributed application data	Embedding storage and similarity retrieval
Data structure	Tables, rows, and columns	Documents, key-value pairs, graphs, wide columns	Vectors, IDs, original content, and metadata
Schema	Usually predefined / structured	Flexible or schema-light	Fixed vector dimensions with flexible metadata
Main query type	Exact matching, joins, ranges, aggregations	Key lookup, document queries, filtering	Semantic similarity, nearest-neighbor search
Typical use cases	Banking, payroll, orders, inventory	User profiles, sessions, logs, content catalogs	RAG, recommendations, semantic search, image search
Examples	PostgreSQL, MySQL, Oracle	MongoDB, DynamoDB, Cassandra	Pinecone, Qdrant, Milvus, Weaviate (PostgreSQL+pgvector = hybrid)

Vector DB vs Vector Store

These two terms sound the same, and there is little technical difference between them — but a "Vector DB" implies a **complete system** (with distribution, sharding, etc.), while a "Vector Store" (e.g. FAISS, ChromaDB) often refers to an in-memory or persistent store used for prototype / POC / MVP work rather than a full-scale production solution.

What's Stored Inside a Vector Database?

ID · Vector/Embedding · Original text or content · Metadata

```
{
  "id": "chunk_101",
  "vector": [0.12, -0.45, 0.88],
  "text": "Employees receive 20 annual leaves.",
  "metadata": {
    "source": "policy.pdf",
    "page": 12,
    "department": "HR"
  }
}
```

Metadata Filtering

Similarity search can be combined with structured filters, e.g.: find semantically similar documents where `department = "HR"` and `year = 2026`. This prevents irrelevant documents from other departments from being retrieved.

CRUD Operations in a Vector DB

Operation	Meaning
Create	Insert vectors
Read	Search / retrieve vectors
Update	Replace embedding or metadata
Delete	Remove vectors

Upsert = Insert if absent, otherwise update.

Collections, Indexes & Namespaces

Different vector databases use different names for essentially the same concept (a logical grouping of vectors):

Database	Term Used
Qdrant	Collection
Pinecone	Index + Namespace
Chroma	Collection
Milvus	Collection
OpenSearch	Index

Vector Dimension

Every embedding model produces a **fixed-size vector**. Example: Model A → 384 dimensions, Model B → 768 dimensions, Model C → 1536 dimensions.

IMPORTANT
The vector database collection/index dimension must match the embedding model's output dimension. A 1536-dimension index cannot directly accept a 768-dimension vector.

Vector Database Evaluation Metrics

- Recall@k
- Precision@k
- Latency
- Throughput
- Memory usage
- Index build time
- Storage cost

Retrievers — Core Concepts

What is a Retriever?

A retriever takes a user query and returns the most relevant documents or chunks from a knowledge source. A retriever normally does **not** generate the final answer itself — it collects the relevant context required to generate the answer.

```
User question
↓
Retriever searches the stored documents
↓
Returns the most relevant chunks
↓
The LLM uses those chunks to generate the answer
```

Simple Example

Suppose a vector database contains 1,000 chunks extracted from PDF documents. The user asks: "What is semantic chunking?" The retriever does **not** send all 1,000 chunks to the LLM — it searches the database and returns only the relevant chunks (e.g. Chunk 12 → definition, Chunk 48 → example, Chunk 91 → advantages), which are then passed to the LLM.

The Librarian Analogy

Real-world Role	RAG Role
Student	User
Library books	Documents
Librarian	Retriever — finds the most relevant information
Teacher	LLM — reads the information and provides the final answer

The student asks a question, the librarian finds the most relevant books/pages and gives them to the teacher, and the teacher reads those pages and generates the final answer. **A retriever does not generate the final answer itself — it finds the most relevant information and passes it to the LLM.**

LangChain Retriever — Code Reminder

```
retriever = vector_store.as_retriever(
    search_type="mmr",
    search_kwargs={
        "k": 4,          # Final number of documents to return
        "fetch_k": 20    # Candidate documents considered by MMR
    }
)

documents = retriever.invoke("What is a vector database?")

for document in documents:
    print(document.page_content)
    print(document.metadata)
    print("-" * 50)
```

Parameter	Meaning
<code>k</code>	Final number of results
<code>filter</code>	Metadata restriction
<code>score_threshold</code>	Minimum acceptable relevance score
<code>fetch_k</code>	Number of candidates fetched before MMR
<code>lambda_mult</code>	Balance between relevance and diversity in MMR

`search_type` decides which search algorithm runs: **similarity** → most similar chunks; **similarity_score_threshold** → only chunks passing the threshold; **mmr** → relevant + diverse chunks. `search_kwargs` configures the selected search algorithm.

Metadata Filtering

A retriever should not rely only on semantic similarity — it should also support structured constraints using document metadata (department, year, document type, source, user role, tenant).

```
metadata = {
  "department": "HR", "year": 2026,
  "document_type": "policy", "access_role": "manager"
}
# User Query: "Show the HR leave policy for 2026."
filter = {"department": "HR", "year": 2026}
```

The retriever searches *only* documents whose metadata matches — even semantically similar documents from other departments/years are ignored.

Common Metadata Filter Types

- Exact-match
- Range filters
- Boolean (AND/OR/NOT)
- Date filters
- Department filters
- Document-type filters
- Source filters
- Tenant filters
- Role-based access filters

Pre-filtering vs Post-filtering

	Pre-filtering	Post-filtering
Order	Filter first → Search later	Search first → Filter later
Flow	All docs → apply metadata filter → allowed docs only → similarity search → final results	All docs → similarity/keyword search → top candidates → apply metadata filter → final allowed results
Example (10,000 docs, HR=1,000)	Filter dept=HR & year=2026 → 150 permitted docs remain → similarity search runs only on those 150	Top-5 semantically similar results fetched first (may include Finance/Legal/Engineering) → then filtered → only 1 (HR, 2026) may remain
Advantages	Smaller search set, better security/tenant isolation, faster, prevents unauthorized docs entering candidate list — preferred for strict authorization	Simpler pipeline for non-security use cases
Limitations	—	May return too few final results; relevant permitted docs may never enter the top-k; unauthorized docs may enter the intermediate candidate set; less suitable for strict access control; larger initial k may be required

Retrieval Types

Type	How it Works	Common Methods / Example
Sparse Retrieval	Searches using exact keywords / term matching (traditional keyword-based). Fails on synonyms, hallucinated similarity, or irrelevant sense mismatches.	BM25, TF-IDF, Keyword Search. Query "employee leave policy" → returns docs containing exact words "employee", "leave", "policy".
Dense Retrieval	Uses embeddings to understand semantic meaning; captures related concepts even when exact keywords differ.	Query "How many days off can employees take?" → retrieves "Employees are entitled to 20 days of annual leave." (BERT / sentence-transformer embeddings)
Hybrid Retrieval	Combines sparse + dense; catches exact keyword matches AND semantically related content.	Query "HR leave policy 2026" → BM25 matches exact terms (HR, leave policy, 2026); dense retrieval finds "employee annual vacation guidelines"; hybrid combines both.

Searching Technique Trade-offs

	Sparse	Dense	Hybrid
Basics	Exact text/keyword frequency matching	ML/embedding-based semantic path	Combination of both

Problem it has	Irrelevant sense keywords match; hallucinated similarity/word frequency issues; misses related concepts (synonyms)	Requires embedding model; slightly slower to compute	Best of both — resolves keyword miss OR semantic miss
Best for	Beginners; exact keyword lookups	AI-application-driven concept retrieval	Production-grade retrieval quality

Query Transformation & Enhancement

CORE PRINCIPLE

In RAG, query quality = answer quality. If a query is weak → the retriever fetches wrong docs → the LLM's answer is also wrong.

Simple rule: Better query → Better chunks → Better answer.

Query transformation improves the user's original query *before* retrieval so the system can find more relevant information. It may rewrite the query, add related terms, or break a complex query into smaller questions.

1. Query Rewriting

Converts an unclear, incomplete, or conversational query into a clearer standalone search query — especially useful in conversational RAG where the current question depends on previous messages.

```
Original query: "What did he say about it?"
↓ (rewritten)
"What did the CEO say about the 2026 acquisition?"
↓
Retriever
```

2. Query Expansion

Original user query ko improve / reformulate karna, taake zyada relevant docs mil sakein. Adds synonyms, related terms, acronyms, spelling variations, or alternative phrases.

When to use: query is short or ambiguous; need to catch synonyms/related words; need to handle spelling variants.

EXAMPLES

Original: "employee leave policy" → Expanded: "employee leave policy OR vacation policy OR annual leave guidelines"

Original: "car" → Expanded: "car, automobile, vehicle"

Original: "Langchain memory" → Enhanced: "Langchain memory modules, conversation memory"

Original: "retrieval" → (expanded with related terms/synonyms)

```
Original query
↓
Add related terms or synonyms
↓
Broader retrieval
```

3. Query Decomposition

Breaks a complex, multi-part question into smaller, simpler sub-questions that can be retrieved and answered separately.

Why needed:

- Complex queries contain multiple concepts
- A retriever alone misses the full context in one shot
- Enables multi-hop reasoning (step-by-step solving)

EXAMPLE

Original: "Compare the revenue of Company A and Company B in 2025 and explain why their growth rates were different."

Sub-query 1: What was Company A's revenue in 2025?

Sub-query 2: What was Company B's revenue in 2025?

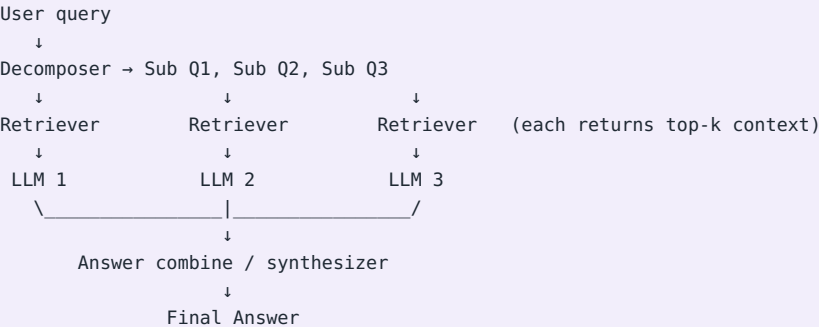
Sub-query 3: What factors affected Company A's growth?

Sub-query 4: What factors affected Company B's growth?

The system retrieves evidence for each sub-query, then combines results to answer the original question.

ANOTHER WORKED EXAMPLE

Original: "What memory modules does LangChain support and how are they different from CrewAI?"
Sub Q1: "What memory modules does LangChain support?"
Sub Q2: "What are CrewAI agents?"
Sub Q3: "LangChain memory vs CrewAI agents — difference?"



DISADVANTAGE

Calls scale with sub-questions — every extra sub-question means another full LLM + Retriever call, so it costs more **time and money**.

Complex query
↓
Multiple smaller sub-queries
↓
Retrieve evidence for each query
↓
Combine the results

4. HyDE — Hypothetical Document Embeddings

Agar query ko directly embed karne ki jagah, pehle LLM se ek hypothetical answer generate karo, aur phir us answer ko embed karke vector store mein search karo.

Why? A full hypothetical answer is often semantically closer to the actual document content than a short user question.

When to use:

- Query बहुत short hai (context kam hai)
- Query aur document mein language mismatch hai
- Answer-style content se search karna behtar ho

EXAMPLE

User asks: "How does Llama 2 improve safety?"
Instead of directly embedding this short query, HyDE first creates something like: "Llama 2 improves safety using supervised safety fine-tuning, RLHF, red teaming, and safety evaluation." Then *this hypothetical answer* is embedded and used for retrieval.

Query
↓
LLM → HyDE (hypothetical answer generated hota hai)
↓
Embedding vector
↓
Retriever → Top-K → LLM → Output

Multi-Query Retriever vs HyDE

Multi-Query Retriever	HyDE
Generates multiple query variations	Generates a single hypothetical answer/document
Searches using several query versions	Searches using the hypothetical-answer embedding
Main goal: improve recall	Main goal: improve semantic matching

MULTI-QUERY EXAMPLE

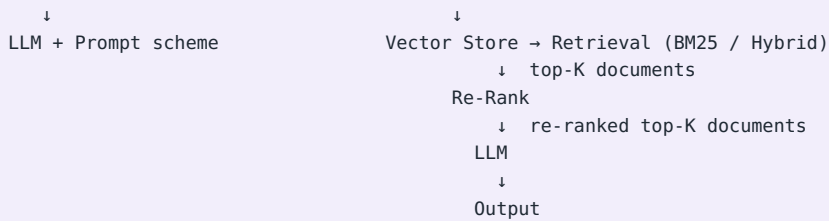
Original: "How does Llama 2 improve safety?" → Variations: (1) "What safety techniques are used in Llama 2?" (2) "How was Llama 2 safety fine-tuned?" (3) "How does Llama 2 reduce unsafe responses?" — each is searched separately, then results are merged and de-duplicated.

Enhanced Query Pipeline (Full Flow)

Enhanced query = vector retrieval (dense search, BM25, MMR combined)

Query → Query Enhancement (LLM + prompt scheme:

synonyms/related terms/broader context add karke)



Key point: query enhancement essentially means explaining/enriching the query for the LLM — adding synonyms, related terms, and broader context.

Reranking, MMR & Result Fusion

Reranking

Reranking is a **second-stage process** that takes documents returned by an initial retriever, scores them more accurately against the user query, and reorders them so the most relevant documents are sent to the LLM.

```
User Query
↓
Initial Retriever (BM25, Vector Search, or Hybrid)
↓
Top Candidate Documents
↓
Reranker
↓
Reordered by Relevance
↓
Top Documents sent to the LLM
```

Why Is Reranking Required?

The initial retriever must search thousands/millions of documents quickly, so it uses a fast method (BM25, vector similarity, hybrid). Fast retrieval gives good candidates, but their original order may not be perfectly accurate. A reranker applies a more powerful (but slower) model only to this smaller candidate set — a practical balance: **Initial Retrieval = fast & broad; Reranking = slower but more accurate.**

WORKED EXAMPLE

Query: "How many days of annual leave do employees receive?"

Initial retriever returns: 1. Remote Work Policy, 2. Sick Leave Policy, 3. Annual Leave Policy, 4. Leave Carry-Forward Policy, 5. Employee Attendance Policy.

Reranker scores each: Annual Leave Policy → 0.95, Leave Carry-Forward → 0.76, Sick Leave → 0.31, Attendance → 0.19, Remote Work → 0.08.

New order: Annual Leave Policy, Leave Carry-Forward Policy, Sick Leave Policy, Employee Attendance Policy, Remote Work Policy. Only the top-ranked documents go to the LLM.

How a Cross-Encoder Reranker Works

A standard embedding retriever encodes the query and documents *separately* (fast — vectors precomputed and stored). A cross-encoder reranker processes the query and **one** candidate document *together*: `[Query + Candidate Document] → Cross-Encoder → Relevance Score`. This joint processing examines detailed word-level relationships but is more computationally expensive since every query-document pair must be processed separately.

Benefits & Limitations

BENEFITS

- Improves ordering of retrieved documents
- Removes weak candidates via a relevance threshold
- Reduces irrelevant context sent to the LLM
- Reduces unnecessary input tokens
- Improves evidence quality
- Works on sparse, dense, or hybrid retrieval results

LIMITATIONS

- Additional latency
- Additional computation
- Additional inference cost

Therefore reranking should normally be applied only to a limited candidate set, not every document in the database.

Real Production Flow

1. User submits a query
2. Apply metadata and security filters
3. Retrieve a broad candidate set (BM25 / vector / hybrid)
4. Send the candidate set to a reranker
5. Calculate query-document relevance scores

- Sort candidates by reranker score
- Apply an optional minimum-score threshold
- Send the best documents to the LLM

Example: 1,000,000 stored chunks → Retriever selects 50 candidates
→ Reranker reorders those 50 → Top 5 chunks passed to the LLM

Reranking was **not** introduced by RAG — it existed in information retrieval before. The 2011 paper "A Cascade Ranking Model for Efficient Ranked Retrieval" formalized multi-stage ranking; 2019's "Passage Re-ranking with BERT" popularized transformer-based semantic reranking. RAG later adopted this same established idea (search → rerank → give best evidence to LLM).

Reranking vs Retrieval

Retrieval	Reranking
Searches the complete collection	Processes only retrieved candidates
Optimized for speed and recall	Optimized for ranking precision
Returns an initial candidate set	Reorders that candidate set
Commonly uses BM25 or embeddings	Commonly uses a cross-encoder / ranking model
First stage	Second stage

Reranking vs Reciprocal Rank Fusion (RRF)

These are **not** the same. RRF combines ranked lists from multiple retrievers using their positions/ranks — it does not need to jointly understand every query-document pair. A semantic (cross-encoder) reranker deeply evaluates the relationship between the query and each candidate. A complete hybrid pipeline may use both: **BM25 + Vector Search → RRF → Cross-Encoder Reranker → Best Context for LLM**.

Main Reranking Approaches

- Pointwise** — each candidate scored independently (Doc A → 0.91, Doc B → 0.67, Doc C → 0.22), then sorted by score.
- Pairwise** — model compares two documents and decides which is more relevant for the query.
- Listwise** — model considers several candidates together and produces an ordered list directly (Input: A,B,C,D → Output: C,A,D,B).

(2019's multi-stage BERT-ranking work describes pointwise **monoBERT** and pairwise **duoBERT** models in a multi-stage architecture.)

Two-Stage Hybrid + Rerank Architecture

- First use a fast retriever (BM25 / FAISS / Hybrid) to fetch top-K documents quickly.
- Then use a slower but more accurate model (Cross-Encoder or LLM) to re-score and reorder those documents by relevance to the query.

```
Query → embedding
      ↓
  Vector Store
    /      \
  Exact    Similar
    ↓        ↓
  BM25    Semantic Search (embedding)
    ↓ top-K    ↓ top-K
    \          /
   Hybrid Score
     ↓ top-K relevant chunk
    Re-Rank
      ↓
    LLM
      ↓
   Output
```

MMR — Maximal Marginal Relevance

MMR first retrieves the most relevant document, then selects the remaining documents by **balancing query relevance and diversity**, avoiding documents too similar to those already selected.

Goal: retrieve relevant documents while avoiding duplicate/highly repetitive documents.

MMR Formula variables: Q = user query · D_i = candidate document · S = documents already selected · $\text{Sim}(D_i, Q)$ = similarity between candidate & query · $\text{Sim}(D_i, D_j)$ = similarity between candidate & already-selected doc · λ (lambda) = controls the balance between relevance and diversity, $\lambda \in [0, 1]$.

- **High λ** → relevance zyada important
- **Low λ** → diversity zyada important

Worked Example ($\lambda = 0.5$, $k = 2$)

Doc	Query Similarity	Similarity with Doc A (selected)	MMR Score
Doc A (selected first)	0.95	—	—
Doc B	0.90	0.95	-0.025
Doc C	0.88	0.40	0.24
Doc D	0.84	0.20	0.32
Doc E	0.80	0.10	0.35

Result: Normal similarity search would pick Doc A, Doc B (nearly identical content). MMR instead picks **Doc A, Doc E** — Doc E is slightly less relevant but provides different information, increasing diversity.

User Query → Retrieve Top 5 Documents → Select Best Document (Doc A)
→ Compare Remaining Documents → (High Query Similarity? + Low Similarity with Doc A?)
→ Choose Next Document

Memory trick: Without MMR — "Python Tutorial Part 1, 2, 3, 4" (near-duplicate). With MMR — "Python Tutorial, Python Project, Python Interview Questions, Python Best Practices" (still relevant, much more diverse).

When to Use / Not Use MMR

Use MMR	Don't Use MMR
RAG mein redundant docs avoid karne ke liye	Sirf precision chahiye (top-1 result kaafi hai)
Chatbots, FAQ apps, document browsers	Docs already diverse hain
Jab retriever बहुत similar results de	LLM se reranking already ho rahi ho

Result Fusion

Used when multiple retrievers return different ranked lists, and we want to combine them into one final ranking.

	Weighted Fusion	RRF (Reciprocal Rank Fusion)
Combines	Raw/normalized scores	Rank positions
Weights	Uses explicit weights	Original RRF does not require weights
Normalization	Score normalization may be needed	Score scales do not need to match

Simple rule: Weighted Fusion = score-based fusion. RRF = rank-based fusion.

1. Weighted Fusion — Worked Example

BM25 score = 0.8, Vector score = 0.9. Weights: BM25 = 0.4, Vector = 0.6.

Final Score = $(0.4 \times 0.8) + (0.6 \times 0.9) = \mathbf{0.86}$

Documents are then sorted by the combined score. Note: scores should be normalized/comparable if retrievers use different scales.

2. Reciprocal Rank Fusion (RRF)

RRF combines *rank positions*, not raw scores. A document appearing near the top in multiple ranked lists gets a higher final

RRF score.

BM25 Ranking: 1. Doc A, 2. Doc B, 3. Doc C

Vector Ranking: 1. Doc B, 2. Doc C, 3. Doc A

→ RRF gives each document points based on its position across both lists, then produces a combined ranking.

Merge Strategy Case Studies (Hybrid Retrieval)

Case	Approach	Notes
1. Without Fusion (Simple Merge)	BM25: A,B,C + Dense: D,B,E → merged as A,B,C,D,E	No proper ranking; BM25 results dominate; dense results may appear too low — not a good production approach.
2. Weighted Fusion	BM25 Score + Dense Score → weighted formula → final ranking	Uses scores
3. RRF	BM25 Rank + Dense Rank → RRF formula → final ranking	Uses ranks
4. Reranking (very common)	BM25 + Dense → merge candidates → Top 50 → Cross-Encoder Reranker → Final Top 5	Reranker decides the final ranking

Advanced Retrieval Patterns

Parent Document Retriever

Search small, return big. It searches using small child chunks, but returns the larger parent document/section that contains the matched chunk.

Large Document → Split into small child chunks → Create embeddings for child chunks
→ User query searches child chunks → Best child chunk found → Return its larger parent section

EXAMPLE

A policy document has a 2,000-token section split into Child Chunk 1, 2, 3 (300 tokens each). User asks: "How many annual leave days are allowed?" Retriever finds Child Chunk 3, but instead of returning only that 300-token chunk, it returns the full parent section (~1,500–2,000 tokens).

Why use it? Small chunks are better for accurate retrieval, while larger parent sections provide better context.

Sentence Window Retriever

Searches using a single sentence (or very small chunk); when it matches, it also returns the **nearby sentences** around it.

Document → Split into sentences → Create embeddings for each sentence
→ User query searches sentences → Best matching sentence found → Return sentence + nearby sentences

EXAMPLE

Sentences 8–12 describe Llama 2's fine-tuning/RLHF pipeline. User asks: "How is Llama 2 aligned using human feedback?" Retriever matches Sentence 10, but returns a window (Sentences 8–12) instead of just Sentence 10.

Why use it? One sentence may be very precise for retrieval but may not contain enough context for the LLM.

Parent Document vs Sentence Window

Parent Document Retriever	Sentence Window Retriever
Search small chunk, return larger parent section	Search one sentence, return nearby sentences

Multi-Hop Retrieval

Retrieve → use the result to retrieve again → combine evidence. Used when one retrieval isn't enough — the system retrieves one piece of info, uses it to form the next query, retrieves again, and continues until it has enough evidence.

User Query → Retrieve first piece of information → Use result to create the next query
→ Retrieve second piece of information → Combine the evidence → Final answer

EXAMPLE

User asks: "Who founded the company that developed Llama 2?"
Hop 1: "Which company developed Llama 2?" → Meta
Hop 2: "Who founded Meta?" → Mark Zuckerberg and co-founders
System combines both hops to answer.

Why use it? Some questions require information from multiple documents or multiple retrieval steps.

Multimodal Retriever

Retrieves relevant information across **different modalities** (text and images), using multimodal embedding models that place compatible modalities in a shared vector space — enabling text-to-image, image-to-text, and image-to-image similarity search.

TEXT-TO-IMAGE

Text Query → Text Encoder
→ Shared Embedding Space
→ Search Image Vectors
→ Relevant Images

IMAGE-TO-IMAGE

Image Query → Image Encoder
→ Image Embedding Space
→ Search Image Vectors
→ Similar Images

Full Retrieval Pipeline Summary

User Query



Query Understanding (Intent Classification, Metadata Extraction, Security/Governance)



Query Transformation (Query Rewriting, Query Expansion, HyDE, Query Decomposition)



(Optional) Query Routing (Vector DB / SQL DB / Knowledge Graph / Web Search / APIs / Multiple Sources)



Retrieval – if routed to a retriever

(Sparse / Dense / Hybrid / Parent Document / Sentence Window / Multi-Hop / Metadata Filtering)



Result Fusion (Weighted Fusion / Reciprocal Rank Fusion)



Reranking



Contextual Compression



LLM

Contextual Compression

Contextual Compression means: first retrieve relevant documents, then **remove the parts that are not useful** for the current query.

User Query → Retriever gets relevant documents → Compressor checks documents against the query
→ Irrelevant text is removed → Only useful context is sent to the LLM

EXAMPLE

Retrieved HR policy chunk (1,000 tokens) covers: leave policy, dress code, WFH rules, travel reimbursement, holiday calendar.
User asks: "How many annual leave days do employees get?"

Contextual compression keeps only: "Employees are entitled to 20 days of annual leave per year. Unused leave can be carried forward up to 5 days." — everything else is removed.

Why use it? Reduces irrelevant context, token usage, LLM cost, and noise in the prompt — and can improve answer quality.

Reranking vs Contextual Compression

Reranking	Contextual Compression
Reorders documents	Removes irrelevant documents / irrelevant parts inside documents
Only the order changes (1000 words stays 1000 words)	The content inside documents changes (1000 words → 120 words)

Easiest way to remember: Retriever finds the documents → Reranker orders them → Contextual Compression trims them.

Step-by-Step Flow

User Query → Base Retriever → Top-K Retrieved Documents
→ Document Compressor → Remove irrelevant content → Compressed Documents → LLM

The `ContextualCompressionRetriever` is a wrapper around an existing retriever. It works in two stages: (1) retrieve relevant documents using any retriever, (2) compress those documents based on the current query before sending to the LLM.

Worked Example — Three Retrieved Chunks

Chunk 1: HR Policy (Leave Policy: 20 days annual leave; Dress Code; Office Timing 9-6)

Chunk 2: Travel Policy (Flight booking, Hotel booking, Taxi reimbursement)

Chunk 3: Medical Insurance (Health, Dental, Vision insurance)

Query: "How many annual leave days are employees entitled to?"

Compressor Output: "Employees receive 20 days of annual leave." — everything else removed.

What Exactly Does the Compressor Do? (3 implementation types)

- LLMChainExtractor** — uses an LLM: Query + Document → LLM → extract only relevant sentences. (e.g., a Leave Policy document with dress code/travel/office timing → compressed to just "20 annual leave days")
- Embeddings Filter** — filters documents/chunks using embedding similarity (no LLM call needed) → faster and cheaper. Retrieved Documents → Embedding Similarity → keep relevant ones → discard the rest.
- Cross-Encoder / Reranker** — a reranker can also act as the compressor: Retrieved Documents → Cross Encoder → remove low-relevance documents; only the most relevant continue through the pipeline.

Why Not Simply Reduce k?

Suppose Retriever → Top 3 Documents, and Document 1 has 1000 tokens but only 20 tokens are actually useful. Reducing `k` will not solve this — you'll still send 1000 tokens to the LLM. Instead, Contextual Compression turns 1000 tokens → 20 useful tokens. This is why it is different from simply retrieving fewer documents.

Production Flow

User Query → Retriever → Top 20 Documents → Contextual Compression
→ Top 20 become only the relevant passages → LLM
(Notice: the NUMBER of documents may stay 20, but each is much shorter)

RAG vs Fine-Tuning

What is RAG? (Recap)

RAG = Retrieval + Augmented Generation — retrieval se knowledge inject karna, without needing to train/fine-tune the model itself.

EXAMPLE

Without RAG: "Black Friday return policy?" → LLM answers only from what it was trained on (generic/imprecise).

With RAG: company's exact policy is fetched from the vector DB → relevant info is found and returned more precisely.

Quick Comparison Table

Column	RAG	Fine-tuning
Main purpose	Gives the model external knowledge/context	Teaches the model new behavior, style, or format
Model weights change?	No	Yes
Cost	Usually lower	Usually higher
Data update	Easy — update documents/vector DB	Harder — retrain or fine-tune again

Detailed Comparison (from notes)

Factor	Prompt Eng.	Fine-tuning	RAG
Cost	Free	Medium	Free
Training	Nahi	Haan	Nahi
Up-to-date info	Nahi	Nahi	Haan
Private data	Haan	Haan	Haan
Expertise	Zero	ML Expert	Low

Best-fit Use Cases

- **Prompt Engineering** → best for quick prototyping, small apps, domain-agnostic small tests
- **Fine-tuning** → best for real-time info, knowledge-base needs, specific domain adaptation, consistent style, customer support with consistent tone
- **RAG** → best for real-time / up-to-date info, large knowledge bases, cost-effective scaling, consistent up-to-date support without retraining

Pros of RAG

- Always up-to-date info
- No training infrastructure required
- Retrieval quality directly affects result quality
- Cost-effective
- Retrieval → best for real-time info, knowledge base, consistent up-to-date support

Pros of Fine-tuning

- Works without retrieval infrastructure
- Context-window limitation is not an issue (knowledge is baked into the weights)
- Fine-tuning → best for specific domain, consistent style, customer support

Real-World Example — "e.g." from notes

Without RAG = Black Friday return policy → creative answer (may hallucinate details).

With RAG = company's DB se exact policy fetch karke precise answer diya jata hai.

3 Steps of RAG (Recap)

1. **Retrieval** → vector DB se relevant docs find karna
2. **Augmentation** → context ka metadata se enrich karna
3. **Generation** → LLM se final answer banana

Core Components of RAG (Recap Diagram)

- 1) Document Ingestion Phase
PDF/Web/DB → Document Splitter → Chunk1, Chunk2, Chunk3 → Embedding Model → Vector Database (FAISS, Pinecone)
- 2) Query Processing Phase
User Query → Query-to-Embedding (some embedding model) → Similarity Search
(cosine similarity, k-NN) → Find most relevant chunks → top-K
- 3) Generation Phase
Augmented Context (query+chunk) → LLM → Generate Response → User

LangChain Quick-Reference

Retriever / Compression Class Map

Concept	Class / Function	Import Statement
MMR	<code>vector_store.as_retriever(search_type="mmr")</code>	No import required — available on any VectorStore
Multi-Query Retriever	<code>MultiQueryRetriever</code>	<code>from langchain_classic.retrievers.multi_query import MultiQueryRetriever</code>
HyDE	<code>HypotheticalDocumentEmbedder</code>	<code>from langchain_classic.chains.hyde.base import HypotheticalDocumentEmbedder</code>
Parent Document Retriever	<code>ParentDocumentRetriever</code>	<code>from langchain_classic.retrievers import ParentDocumentRetriever</code>
Hybrid Retrieval	<code>EnsembleRetriever</code> + <code>BM25Retriever</code> + Vector Retriever	<code>from langchain_classic.retrievers import EnsembleRetriever</code> <code>from langchain_community.retrievers import BM25Retriever</code>
Reciprocal Rank Fusion	<code>EnsembleRetriever</code> (internally <code>weighted_reciprocal_rank()</code>)	<code>from langchain_classic.retrievers import EnsembleRetriever</code>
Weighted Fusion (Weighted RRF)	<code>EnsembleRetriever(weights=[...])</code>	<code>from langchain_classic.retrievers import EnsembleRetriever</code>
Contextual Compression	<code>ContextualCompressionRetriever</code>	<code>from langchain_classic.retrievers.contextual_compression import ContextualCompressionRetriever</code>
LLM-based Compression	<code>LLMChainExtractor</code>	<code>from langchain_classic.retrievers.document_compressors import LLMChainExtractor</code>
Embedding-based Filtering	<code>EmbeddingsFilter</code>	<code>from langchain_classic.retrievers.document_compressors import EmbeddingsFilter</code>
Cross-Encoder Reranking	<code>CrossEncoderReranker</code>	<code>from langchain_classic.retrievers.document_compressors import CrossEncoderReranker</code>
Cross-Encoder Model	<code>HuggingFaceCrossEncoder</code>	<code>from langchain_community.cross_encoders import HuggingFaceCrossEncoder</code>
LLM Listwise Reranking	<code>LLMListwiseRerank</code>	<code>from langchain_classic.retrievers.document_compressors import LLMListwiseRerank</code>
Sentence Window Retrieval	No dedicated built-in retriever	Custom implementation (metadata + custom retrieval logic)
Multi-Hop Retrieval	No dedicated <code>MultiHopRetriever</code>	Implement using LangGraph / iterative retriever calls
Graph Traversal Retrieval	<code>GraphVectorStoreRetriever</code>	<code>from langchain_community.graph_vectorstores import GraphVectorStoreRetriever</code>

Selected Code Snippets

```
# 1. MMR
retriever = vector_store.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 4, "fetch_k": 20, "lambda_mult": 0.5}
)

# 2. Multi-Query Retriever
from langchain_classic.retrievers.multi_query import MultiQueryRetriever
multi_query_retriever = MultiQueryRetriever.from_llm(
```

```

    retriever=vector_store.as_retriever(), llm=llm, include_original=True
)
docs = multi_query_retriever.invoke("How does Llama 2 improve safety?")

# 3. HyDE
from langchain_classic.chains.hyde.base import HypotheticalDocumentEmbedder
from langchain_openai import OpenAI, OpenAIEmbeddings
base_embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
hyde_embeddings = HypotheticalDocumentEmbedder.from_llm(
    llm=OpenAI(), base_embeddings=base_embeddings, prompt_key="web_search"
)

# 4. Parent Document Retriever
from langchain_classic.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore
from langchain_text_splitters import RecursiveCharacterTextSplitter
parent_splitter = RecursiveCharacterTextSplitter(chunk_size=2000)
child_splitter = RecursiveCharacterTextSplitter(chunk_size=300)
store = InMemoryStore()
retriever = ParentDocumentRetriever(
    vectorstore=vector_store, docstore=store,
    child_splitter=child_splitter, parent_splitter=parent_splitter
)
retriever.add_documents(documents)

# 5. Hybrid Retrieval
from langchain_community.retrievers import BM25Retriever
from langchain_classic.retrievers import EnsembleRetriever
bm25 = BM25Retriever.from_documents(chunks); bm25.k = 5
dense = vector_store.as_retriever(search_kwargs={"k": 5})
hybrid = EnsembleRetriever(retrievers=[bm25, dense], weights=[0.5, 0.5])

# 6. Contextual Compression (LLM Extraction)
from langchain_classic.retrievers import ContextualCompressionRetriever
from langchain_classic.retrievers.document_compressors import LLMChainExtractor
compressor = LLMChainExtractor.from_llm(llm)
compression_retriever = ContextualCompressionRetriever(
    base_retriever=dense_retriever, base_compressor=compressor
)

# 7. Embedding Filter Compression
from langchain_classic.retrievers.document_compressors import EmbeddingsFilter
embeddings_filter = EmbeddingsFilter(embeddings=embeddings, similarity_threshold=0.75)

# 8. Cross-Encoder Reranking
from langchain_classic.retrievers.document_compressors import CrossEncoderReranker
from langchain_community.cross_encoders import HuggingFaceCrossEncoder
model = HuggingFaceCrossEncoder(model_name="cross-encoder/ms-marco-MiniLM-L6-v2")
reranker = CrossEncoderReranker(model=model, top_n=5)

```

IMPORTANT NOTE

LangChain does **not** currently expose a canonical built-in `SentenceWindowRetriever` class (unlike LlamaIndex's `SentenceWindowNodeParser` pattern) — it must be implemented via custom metadata + retrieval logic. Similarly, there is no universal `MultiHopRetriever` class; general multi-hop retrieval is implemented using LangGraph + Retriever + LLM + state, calling `retriever.invoke()` iteratively across hops.

Interview Preparation — Q&A

All answers below are derived strictly from the concepts covered in the preceding sections.

RAG Fundamentals

1 What is RAG and what do the three letters stand for?

RAG = Retrieval-Augmented Generation. It's a technique that helps an LLM answer questions using external data (PDFs, documents, websites, databases, knowledge bases) by giving relevant context to the LLM before it answers — instead of relying only on what the model learned during training.

2 Break down the three components of RAG.

Retrieval = finding relevant information from documents. **Augmentation** = adding that retrieved info as extra context to the user's question (improving the LLM's input). **Generation** = the LLM produces the final answer using that context.

3 Why do we need RAG — what problems does it solve?

Four core problems: (1) LLMs don't have private/company data, (2) LLMs may not have the latest data (training cutoff), (3) LLMs can hallucinate — generate confident but wrong answers, and RAG grounds answers in retrieved context to reduce this, (4) fine-tuning isn't practical for frequently changing data — e.g. if an HR policy changes monthly, update the RAG documents/vector DB instead of re-training the model.

4 Walk through the RAG pipeline end-to-end.

Documents → Parsing/Loading → Chunking → Embeddings → Vector Store → (User query) → Query Embedding → Similarity Search → Retriever returns top-K chunks → Prompt = Question + Retrieved Context (Augmentation) → LLM → Final Answer.

5 List the "moving components" of a RAG system and explain what a moving component is.

A moving component is any part whose output directly contributes to the end result. RAG's 10 moving components: Data Parsing, Chunking, Embedding, Vector DB, LLM, Graph RAG (alternative), Retriever, Prompt arrangement, choice of LLM, Output Parser. This is why RAG "seems simple" but has many parts that must each work correctly.

6 Why is a vector database required instead of a traditional database for RAG?

Traditional databases perform exact matching only. A vector database converts data into embeddings so meaning-based matches can be found — e.g., searching "cat" can retrieve "kitten" because their embedding vectors are close in meaning, which a traditional exact-match DB cannot do.

RAG vs Fine-Tuning

7 How is RAG different from fine-tuning?

RAG gives the model external knowledge/context without changing model weights — it's usually lower cost and data updates are easy (just update the documents/vector DB). Fine-tuning teaches the model new behavior, style, or format by actually changing model weights — it's usually higher cost, and updating knowledge requires retraining/fine-tuning again.

8 When would you choose fine-tuning over RAG, and vice versa?

Choose **fine-tuning** for a specific domain, consistent style/tone (e.g. customer support), or when you can't rely on retrieval infrastructure and context-window limits are a concern. Choose **RAG** for real-time/up-to-date information, large knowledge bases, cost-effective scaling, and cases where the underlying data changes frequently.

9 Give a concrete example illustrating the difference between answering with and without RAG.

Without RAG: asking about a "Black Friday return policy" gets a generic, possibly hallucinated answer from the model's training data. With RAG: the exact policy document is fetched from the vector DB, giving a precise, grounded answer.

Data Parsing

10 What are the three perspectives from which data can be understood?

(1) Nature/Modality — text, image, audio, video. (2) Storage Format — PDF, DOCX, CSV, JSON, YAML, MP3, MP4. (3) Structure — structured (tables/CSV/SQL), semi-structured (JSON/YAML/HTML), unstructured (PDFs/images/audio/video/reports).

11 Is LangChain a parsing library?

No. LangChain is not primarily a parsing library — it mainly provides a common wrapper/interface over other parsing libraries (PyPDF, PyMuPDF, pdfplumber, python-docx, Unstructured, etc.). It gives a standard Document format (page_content + metadata) so parsed content can plug easily into the RAG pipeline.

Chunking

12 What is chunking and why is it needed?

Chunking divides large data into smaller, meaningful units so each piece can be embedded, stored, and retrieved independently. It's needed because of LLM context-window limits, to improve retrieval accuracy (small relevant sections beat one huge mixed-topic document), to lower token usage/cost, for faster processing, and to send only relevant information to the LLM.

13 Is chunking always required? Give examples of when it can be skipped.

No — chunking is a design choice, not compulsory. It can be skipped when input is already short, a database record is already a complete meaningful unit, SQL can retrieve exact rows directly, the whole document comfortably fits the model's context window, or data is already separated into FAQs/tickets/products. Example: one FAQ Q&A pair is already an atomic unit. However, a 200-page policy PDF generally requires chunking.

14 What is chunk size and chunk overlap, and why are they called hyperparameters?

Chunk size = the maximum target content length per chunk (e.g. 300 characters/tokens). Chunk overlap = the amount of repeated content between neighboring chunks, which helps preserve meaning when an important sentence falls near a boundary. They're hyperparameters because there's no single "correct" value — it must be evaluated experimentally using real questions and retrieval results.

15 What's the trade-off between small and large chunks?

Smaller chunks improve retrieval precision but may lose surrounding context/explanation. Larger chunks preserve more context but can reduce retrieval precision (may contain irrelevant info mixed in). Excessive overlap increases duplication and storage/token cost.

16 Explain how RecursiveCharacterTextSplitter works.

It splits text recursively using multiple separators in priority order: first tries paragraph boundaries ("
"), then line boundaries ("
") if a paragraph is too large, then sentence boundaries (". ") if a line is still too large, then splits between words (" "), and finally falls back to individual characters (""). Parameters: chunk_size (max chars per chunk), chunk_overlap (repeated chars between chunks), length_function (how size is measured), add_start_index (stores original position as metadata).

17 What is semantic chunking and what problem does it solve?

Problem: naive/fixed-size chunking (e.g. cutting every 500 tokens) blindly splits text without regard to meaning, potentially breaking a concept mid-thought. Solution: semantic chunking cuts based on where the meaning changes — it segments the document into sentences, embeds each sentence, computes similarity between adjacent sentences, merges similar adjacent sentences into the same chunk, and places a boundary where similarity drops (topic change).

18 How should tables be chunked, and what must be preserved?

Tables can be chunked by rows (with overlapping rows for related records), by category (group-by, often more meaningful than arbitrary row splits), or by column (with a common key like Order_ID preserved to keep chunks connected). Every table chunk must preserve column names, primary/joining keys, table title, units, and metadata — otherwise raw values become meaningless (e.g. "O-101, Laptop, 2, 2400" vs. a labeled version).

19 When should you use SQL instead of embedding a table for vector retrieval?

When data is already in a SQL database and the user wants exact calculations, filters, joins, totals, or averages (e.g. "What is the

total revenue?", "Show all orders from the North region") — SQL is the better solution. Vector retrieval is more useful when the table comes from a PDF, isn't in a database, or the user asks meaning-based/semantic questions.

20 Name common mistakes made while chunking.

Choosing chunk size without evaluating retrieval; chunks too small or too large; excessive overlap; splitting headings away from their content; splitting tables without preserving headers; splitting code mid-function/class; removing useful metadata; using semantic/agentive chunking without comparing a simpler baseline; assuming one strategy works for every data type.

Vector Databases

21 What is a vector database and how does it differ from SQL/NoSQL?

A vector database stores data as numerical embeddings and retrieves the most similar ones via similarity search — used for semantic search, RAG, recommendations, and image search. SQL databases do exact matching/joins/aggregations on structured tables; NoSQL handles flexible/distributed application data (key-value, documents); vector DBs specialize in embedding storage and nearest-neighbor/semantic similarity retrieval.

22 What is a vector index, and name three examples.

A vector index is a data structure that organizes embeddings so similar vectors can be found quickly without comparing the query to every stored vector. Examples: Flat, IVF, HNSW. The vector database stores the vectors; the index makes searching them fast.

23 Explain the difference between Exact Nearest Neighbor Search and Approximate Nearest Neighbor (ANN) Search.

Exact search compares the query against every stored vector — 100% accurate but slower on large datasets; best for small datasets/evaluation (uses a Flat index). ANN searches only a selected part of the index to return close matches much faster, but may occasionally miss the true best result; best for large-scale production search (uses IVF or HNSW).

24 How does IVF indexing work?

IVF (Inverted File Index) clusters all vectors into groups — similar data ends up in the same cluster. A query's embedding first finds the nearest cluster(s), and the search runs only within that cluster, reducing the search space and producing faster results (an approximate/clustering-based method).

25 How does HNSW work?

HNSW (Hierarchical Navigable Small World) is a graph-based, multi-layer structure. A query starts navigation at the sparse top layer and moves down through denser layers following graph edges toward the nearest neighbors — a non-linear, approximate search much faster than scanning every vector.

26 What are the three similarity metrics used in vector search, and when is each preferred?

Cosine Similarity measures the angle/direction between vectors (higher = more similar; most common for text embeddings). Euclidean Distance (L2) measures straight-line distance (lower = more similar). Dot Product multiplies and sums corresponding values (higher = more similar; equivalent to cosine similarity when vectors are normalized).

27 What is stored inside a vector database record?

ID, the Vector/Embedding, the original text/content, and Metadata (e.g. source file, page number, department) — this allows both similarity search and metadata filtering.

28 Explain CRUD and Upsert in the context of a vector DB.

Create = insert vectors, Read = search/retrieve vectors, Update = replace embedding or metadata, Delete = remove vectors. Upsert = insert if the record is absent, otherwise update it.

29 Why must the vector DB index dimension match the embedding model's output dimension?

Every embedding model produces a fixed-size vector (e.g. 384, 768, or 1536 dimensions). A vector database collection/index is built for a specific dimensionality, so a 1536-dimension index cannot directly accept a 768-dimension vector — dimensions must match exactly.

30 Name key vector database evaluation metrics.

Recall@k, Precision@k, Latency, Throughput, Memory usage, Index build time, Storage cost.

Retrievers

31 What is a retriever, and does it generate the final answer?

A retriever takes a user query, searches a knowledge source, and returns the most relevant documents/chunks. It does NOT generate the final answer itself — it collects the relevant context, which the LLM then uses to produce the answer (the librarian-vs-teacher analogy: retriever = librarian who finds the pages, LLM = teacher who reads them and answers).

32 What do the parameters `k`, `fetch_k`, `filter`, `score_threshold`, and `lambda_mult` control in a LangChain retriever?

`k` = final number of results returned. `fetch_k` = number of candidates fetched before MMR is applied. `filter` = metadata restriction. `score_threshold` = minimum acceptable relevance score. `lambda_mult` = balances relevance vs diversity in MMR.

33 Differentiate sparse, dense, and hybrid retrieval.

Sparse retrieval uses exact keyword/term matching (BM25, TF-IDF) — fast but misses synonyms/semantically related content. Dense retrieval uses embeddings to capture semantic meaning, retrieving relevant content even with different wording. Hybrid retrieval combines both, catching exact keyword matches and semantically related content — generally best for production.

34 What is metadata filtering, and why combine it with similarity search?

Metadata filtering restricts retrieval using structured fields (department, year, document type, role, tenant) alongside semantic similarity. This prevents irrelevant or unauthorized documents — even if semantically similar — from being retrieved, e.g. restricting results to `department="HR"` and `year=2026`.

35 Explain pre-filtering vs post-filtering, and when each is preferred.

Pre-filtering applies metadata conditions BEFORE the similarity search (filter first, then search a smaller allowed set) — preferred for strict authorization since it prevents unauthorized documents from ever entering the candidate list, improves speed, and supports tenant isolation. Post-filtering searches first, then filters the top candidates afterward — it can return too few final results if relevant documents don't make the initial top-k, and is less suitable for strict access control.

Query Transformation & Enhancement

36 Why does query quality matter so much in RAG?

In RAG, query quality directly determines answer quality: a weak query causes the retriever to fetch the wrong documents, which causes the LLM to generate a wrong answer. Simple rule: Better query → Better chunks → Better answer.

37 What are the three main query enhancement techniques?

(1) Query Expansion — reformulating/adding synonyms and related terms to the original query. (2) Query Decomposition — breaking a complex multi-part question into smaller sub-questions. (3) HyDE — generating a hypothetical answer first and embedding that instead of the raw query.

38 When would you use Query Expansion?

When the query is short or ambiguous, when synonyms/related words need to be captured, or when spelling variants need handling. Example: "employee leave policy" → "employee leave policy OR vacation policy OR annual leave guidelines".

39 Walk through an example of Query Decomposition and its main drawback.

A complex query like "Compare Company A and B's 2025 revenue and explain why growth rates differed" is broken into sub-questions (A's revenue, B's revenue, factors for A's growth, factors for B's growth); each is retrieved and answered separately, then combined by a synthesizer. Drawback: cost and time scale with the number of sub-questions, since each requires its own retriever + LLM call.

40 What is HyDE and why does it help?

HyDE (Hypothetical Document Embeddings): instead of directly embedding a short user query, an LLM first generates a hypothetical answer/document, and THAT hypothetical text is embedded and used to search the vector store. It helps because a full hypothetical answer is often semantically closer to actual document content than a short question — useful when the query is very short, or when there's a language/style mismatch between query and documents.

41 Differentiate Multi-Query Retriever from HyDE.

Multi-Query Retriever generates multiple variations of the original query, searches with each, and merges/de-duplicates results — its main goal is improving recall. HyDE generates a single hypothetical answer, embeds that, and searches with it — its main goal is improving semantic matching.

Reranking, MMR & Fusion

42 What is reranking and why is a second stage needed after retrieval?

Reranking is a second-stage process that takes an initial retriever's candidates, scores them more accurately against the query using a slower but more precise model (e.g. cross-encoder), and reorders them. It's needed because the first-stage retriever must be fast across huge collections (BM25/vector/hybrid), so its ordering may not be perfectly accurate — reranking applies a more expensive model only to the small candidate set as a practical fast+accurate balance.

43 How does a cross-encoder reranker differ from a standard embedding retriever?

A standard retriever encodes the query and documents separately (fast, since document vectors are precomputed). A cross-encoder processes the query and ONE candidate document together as a joint input, producing a more accurate relevance score by examining detailed word-level relationships — but it's more computationally expensive since every query-document pair must be scored individually.

44 Is reranking a RAG invention?

No. Reranking predates RAG — multi-stage/cascade ranking existed in traditional search systems (formalized in the 2011 "Cascade Ranking Model" paper), and 2019's "Passage Re-ranking with BERT" popularized transformer-based semantic reranking. RAG later adopted the same idea: search first, then rerank, then give the best evidence to the LLM.

45 Differentiate Pointwise, Pairwise, and Listwise reranking.

Pointwise: each candidate is scored independently, then sorted (e.g. monoBERT). Pairwise: the model compares two documents at a time to decide which is more relevant (e.g. duoBERT). Listwise: the model considers several candidates together and produces an ordered list directly.

46 Is Reciprocal Rank Fusion (RRF) the same as reranking?

No. RRF combines multiple ranked lists using rank positions only — it doesn't jointly analyze query-document relevance. A semantic/cross-encoder reranker deeply evaluates the query-document relationship. A production pipeline might use both together: BM25 + Vector → RRF → Cross-Encoder Reranker → best context for the LLM.

47 What problem does MMR (Maximal Marginal Relevance) solve, and how?

Problem: plain top-k similarity search often returns redundant/near-duplicate documents. MMR solves this by selecting the most relevant document first, then choosing subsequent documents by balancing relevance to the query AND low similarity to already-selected documents — controlled by λ (high λ favors relevance, low λ favors diversity).

48 When should you NOT use MMR?

When you only need precision and the top-1 result is sufficient, when the documents are already diverse, or when an LLM/reranker downstream already handles reordering/deduplication.

49 Differentiate Weighted Fusion from Reciprocal Rank Fusion.

Weighted Fusion combines raw/normalized scores from multiple retrievers using explicit weights (score-based fusion) — scores may need normalization first. RRF combines rank positions instead of scores (rank-based fusion) and doesn't require explicit weights or matching score scales.

50 Why is "simple merge" of hybrid retrieval results a poor production approach?

Simply concatenating BM25 and dense results gives no proper ranking logic — BM25 results tend to dominate and dense results

can appear too low in the list. Production systems instead use Weighted Fusion, RRF, or (most commonly) send merged candidates through a cross-encoder reranker to decide the final order.

Advanced Patterns & Contextual Compression

51 Explain the Parent Document Retriever pattern.

"Search small, return big." It searches using small child chunks (good for retrieval precision) but returns the larger parent section containing the matched chunk (good for context). Example: a 2,000-token policy section split into 300-token child chunks — the retriever finds the best matching child chunk but returns the entire parent section to the LLM.

52 Explain the Sentence Window Retriever, and how it differs from Parent Document Retriever.

Sentence Window Retriever searches using a single sentence, then returns that sentence PLUS nearby sentences (a window) for extra context. Parent Document Retriever returns a larger parent SECTION, while Sentence Window returns nearby SENTENCES — both trade off precise matching against sufficient surrounding context, just at different granularities.

53 What is multi-hop retrieval and when is it needed?

Multi-hop retrieval retrieves one piece of information, uses it to form the next query, retrieves again, and repeats until there's enough evidence to answer — needed when a question requires chaining information across multiple documents/steps (e.g. "Who founded the company that developed Llama 2?" requires first finding the company, then its founders).

54 What is Contextual Compression and how does it differ from Reranking?

Contextual Compression retrieves documents, then removes the parts NOT useful for the current query — reducing token count while keeping the same number of documents (e.g. 1000 tokens → 20 useful tokens). Reranking only changes the ORDER of documents; the content stays the same length. Memory trick: Retriever finds documents → Reranker orders them → Contextual Compression trims them.

55 Why doesn't simply reducing k solve the "too much irrelevant content" problem?

Reducing k only reduces the NUMBER of documents retrieved — it doesn't address that any single retrieved document may be mostly irrelevant (e.g. 1000 tokens with only 20 useful ones). You'd still send the full 1000 tokens to the LLM. Contextual compression instead trims the CONTENT within each document, which reducing k cannot do.

56 Name the three ways a Document Compressor can work.

(1) LLMChainExtractor — uses an LLM to extract only query-relevant sentences from each document. (2) Embeddings Filter — filters documents using embedding similarity without an LLM call (faster/cheaper). (3) Cross-Encoder/Reranker acting as compressor — removes low-relevance documents using cross-encoder scores.

57 What is a multimodal retriever?

A retriever that finds relevant information across different modalities (text and images) using multimodal embedding models that place compatible modalities into a shared vector space — enabling text-to-image, image-to-text, and image-to-image similarity search.

58 Describe the full production RAG retrieval pipeline in order.

User Query → Query Understanding (intent classification, metadata extraction, security/governance) → Query Transformation (rewriting, expansion, HyDE, decomposition) → (Optional) Query Routing (vector DB, SQL, knowledge graph, web search, APIs) → Retrieval (sparse/dense/hybrid/parent-doc/sentence-window/multi-hop + metadata filtering) → Result Fusion (weighted fusion or RRF) → Reranking → Contextual Compression → LLM.